DAS 839 - NoSQL Systems

Assignment II

MapReduce & Apache Hadoop

Group Members:	Nishant Khatri Arshbir Singh Dang Manav Jindal Siddharth Goswami
Roll Numbers:	IMT2023009 IMT2023132 IMT2023535 IMT2023542

Dataset:	Wikipedia-EN-20120601_ARTICLES (10,000 articles)
Platform:	Apache Hadoop (Single Node, macOS)

April 14, 2026

Contents

1 Problem 1: Co-Occurring Word Matrix Generation 2
1.1 Part (a): Top 50 Most Frequent Words . . . 2
1.1.1 Program Logic . . . 2
1.1.2 Pseudocode . . . 2
1.1.3 Key Design Decisions . . . 2
1.1.4 Top 50 Words (Full Dataset) . . . 2
1.1.5 Impact of Stopword Filtering . . . 3
1.1.6 HDFS Commands . . . 4
1.2 Part (b): Co-Occurrence Matrix — Pairs Approach . . . 4
1.2.1 Program Logic . . . 4
1.2.2 Pseudocode . . . 5
1.2.3 Key Design Decisions . . . 5
1.2.4 Runtime Results . . . 5
1.2.5 HDFS Commands . . . 5
1.3 Part (c): Co-Occurrence Matrix — Stripes Approach . . . 6
1.3.1 Program Logic . . . 6
1.3.2 Pseudocode . . . 6
1.3.3 Pairs vs Stripes Comparison . . . 6
1.3.4 Runtime Results . . . 7
1.4 Part (e): Local Aggregation . . . 7
1.4.1 Overview . . . 7
1.4.2 Map-Function Level Pseudocode . . . 7
1.4.3 Map-Class Level Pseudocode . . . 7
1.4.4 Runtime Comparison — Pairs with Local Aggregation . . . 8
1.4.5 Runtime Comparison — Stripes with Local Aggregation . . . 8
1.4.6 Analysis . . . 8

2 Problem 2: Indexing Documents via Hadoop 9
2.1 Part (a): Document Frequency . . . 9
2.1.1 Program Logic . . . 9
2.1.2 Pseudocode . . . 9
2.1.3 Key Design Decisions . . . 9
2.1.4 Top 100 Terms by Document Frequency . . . 10
2.1.5 Runtime . . . 10
2.1.6 HDFS Commands . . . 10
2.2 Part (b): TF-IDF Score . . . 11
2.2.1 Program Logic . . . 11
2.2.2 Formula . . . 11
2.2.3 Pseudocode . . . 11
2.2.4 Key Design Decisions . . . 11
2.2.5 Sample Output . . . 12
2.2.6 Runtime . . . 12
2.2.7 HDFS Commands . . . 12

3 Overall Runtime Summary 12

4 Conclusions 13

1 Problem 1: Co-Occurring Word Matrix Generation

1.1 Part (a): Top 50 Most Frequent Words

1.1.1 Program Logic

The program `TopWordsPairs.java` identifies the top 50 most frequently occurring non-stopword words across all 10,000 Wikipedia articles using a standard MapReduce word count approach with distributed caching for stopword filtering.

1.1.2 Pseudocode

Listing 1: TopWordsPairs Mapper

```
setup(context):
    Load stopwords from distributed cache into Set<stopWords>

map(key, line, context):
    line = line.toLowerCase()
    line = removeNonAlphabetic(line)
    tokens = line.split(whitespace)
    for token in tokens:
        if len(token) > 2 AND token NOT in stopWords:
            emit(token, 1)

reduce(word, counts, context):
    sum = 0
    for count in counts:
        sum += count
    emit(word, sum)
```

1.1.3 Key Design Decisions

- Used `CustomFileInputFormat` (instructor-provided) to treat each Wikipedia article as one input record.
- Used `setCombinerClass(IntSumReducer.class)` for local aggregation before the shuffle phase, reducing network traffic.
- Stopwords loaded via Hadoop distributed cache (`-skippatterns` flag), consistent with the `WordCount2.java` example.
- Words shorter than 3 characters are filtered to remove noise.

1.1.4 Top 50 Words (Full Dataset)

After running on all 10,000 articles, the top 50 most frequent non-stopword words are:

Rank	Word	Count	Rank	Word	Count
1	first	42276	26	south	16700
2	city	33146	27	made	16563
3	two	32565	28	population	16106
4	american	29215	29	january	16100
5	united	28849	30	september	15642
6	time	28369	31	press	15566
7	county	27917	32	area	15457
8	world	27485	33	october	15340
9	university	27048	34	film	15272
10	states	26856	35	july	15259
11	years	24147	36	high	15195
12	people	24062	37	through	15108
13	history	23324	38	june	15096
14	state	23109	39	town	15046
15	war	23095	40	life	14893
16	school	20039	41	including	14881
17	national	19814	42	early	14804
18	york	19687	43	december	14795
19	john	19318	44	century	14601
20	year	19188	45	both	14530
21	news	17788	46	november	14409
22	age	17521	47	march	17042
23	org	17388	48	april	16965
24	north	17022	49	part	16871
25	known	16807	50	three	16733

Table 1: Top 50 most frequent non-stopword words across 10,000 Wikipedia articles

1.1.5 Impact of Stopword Filtering

Before applying stopword filtering, the raw word counts were dominated by HTML artifacts and markup tokens from the Wikipedia dataset. The table below compares the top tokens before and after filtering:

Token	Raw Count	Type	Status After Filtering
apos	273,057	HTML entity (`'`)	**Removed**
quot	268,016	HTML entity (`"`)	**Removed**
http	79,117	URL prefix	**Removed**
category	60,536	Wikipedia markup	**Removed**
this	58,767	Stopword	**Removed**
www	54,691	URL prefix	**Removed**
other	35,814	Stopword	**Removed**
used	23,497	Stopword	**Removed**
html	19,928	Markup artifact	**Removed**
lt	16,523	HTML entity (`<`)	**Removed**
first	42,276	Meaningful English word	**Kept**
city	33,146	Meaningful English word	**Kept**
american	29,215	Meaningful English word	**Kept**

Table 2: Effect of stopword filtering — markup artifacts removed, meaningful words retained

Observation: Without filtering, the top 2 tokens (`apos` and `quot`) would have counts of 273k and 268k respectively — far exceeding any real content word. These are HTML encoding artifacts present throughout the Wikipedia dump. The stopword list effectively removes these along with URL tokens (`http`, `www`), Wikipedia markup (`category`, `html`), and common stopwords (`this`, `other`, `used`). The minor artifact `org` (17,388) remains in the filtered output — this is acceptable as it occurs in legitimate Wikipedia article URLs and organization names.

1.1.6 HDFS Commands

```
# Upload stopwords to HDFS
hdfs dfs -put stopwords.txt /input/stopwords.txt

# Run job
hadoop jar TopWordsPairs.jar TopWordsPairs \
    /wiki/input /wiki/output1a \
    -skippatterns /input/stopwords.txt

# Extract top 50 words sorted by count
hdfs dfs -cat /wiki/output1a/part-r-00000 \
    | sort -t$'\t' -k2 -rn | head -50 | tee top50words.txt

# Extract only word names (no counts)
awk '{print␣$1}' top50words.txt > top50words_only.txt
```

1.2 Part (b): Co-Occurrence Matrix — Pairs Approach

1.2.1 Program Logic

`CoOccurrencePairs.java` builds a co-occurrence matrix using the PAIRS approach. For each pair of top-50 words appearing within a window of distance d, it emits one key-value

pair per co-occurrence.

1.2.2 Pseudocode

Listing 2: CoOccurrencePairs Mapper

```
setup(context):
    Load stopWords from local file
    Load top50Words from local file
    d = conf.getInt("cooccurrence.distance", 1)

map(key, line, context):
    words = [w for w in line if w in top50Words AND w not in
       stopWords]
    for i in range(len(words)):
        for j in range(i+1, min(i+d+1, len(words))):
            emit("(" + words[i] + "," + words[j] + ")", 1)
            emit("(" + words[j] + "," + words[i] + ")", 1)

reduce(pair, counts, context):
    emit(pair, sum(counts))
```

1.2.3 Key Design Decisions

- Only top-50 words are considered — all other words are filtered out before pair generation.
- Pairs are emitted in both directions: (A, B) and (B, A) to build a symmetric matrix.
- A Combiner is used for local partial aggregation.
- Distance d is passed as a command-line argument (`-d`).

1.2.4 Runtime Results

Distance (d)	Runtime
1	5m 19s
2	5m 56s
3	7m 10s
4	9m 58s

Table 3: CoOccurrencePairs runtime for varying distances on 10,000 Wikipedia articles

Observation: Runtime increases with d because larger windows generate exponentially more pairs per word, increasing both map output size and reducer workload.

1.2.5 HDFS Commands

```
time hadoop jar CoOccurrencePairs.jar CoOccurrencePairs \
    /wiki/input /wiki/output1b_d1 -d 1
```

```
time hadoop jar CoOccurrencePairs.jar CoOccurrencePairs \
    /wiki/input /wiki/output1b_d2 -d 2
time hadoop jar CoOccurrencePairs.jar CoOccurrencePairs \
    /wiki/input /wiki/output1b_d3 -d 3
time hadoop jar CoOccurrencePairs.jar CoOccurrencePairs \
    /wiki/input /wiki/output1b_d4 -d 4
```

1.3 Part (c): Co-Occurrence Matrix — Stripes Approach

1.3.1 Program Logic

`CoOccurrenceStripes.java` builds the same co-occurrence matrix but using the STRIPES approach. Instead of emitting one key-value pair per co-occurrence, each word emits a single `MapWritable` containing all its neighbors and their counts.

1.3.2 Pseudocode

Listing 3: CoOccurrenceStripes Mapper

```
map(key, line, context):
    words = [w for w in line if w in top50Words]
    for i in range(len(words)):
        stripe = MapWritable()
        for j in neighbors of i within distance d:
            stripe[words[j]] += 1
        emit(words[i], stripe)

reduce(word, stripes, context):
    merged = {}
    for stripe in stripes:
        for neighbor, count in stripe:
            merged[neighbor] += count
    emit(word, merged)
```

1.3.3 Pairs vs Stripes Comparison

Aspect	Pairs	Stripes
Emissions per word	One per neighbor	One per word (map of neighbors)
Key size	Small (word pair string)	Small (single word)
Value size	Small (integer 1)	Large (MapWritable)
Shuffle volume	High	Lower
Reducer complexity	Simple sum	Map merge

Table 4: Pairs vs Stripes design comparison

1.3.4 Runtime Results

Distance (d)	Runtime
1	11m 41s
2	10m 00s
3	8m 57s
4	8m 44s

Table 5: CoOccurrenceStripes runtime for varying distances on 10,000 Wikipedia articles

Observation: Interestingly, Stripes runtime *decreases* as d increases. This is because larger d means more neighbors per stripe, but the number of emitted keys stays the same (one per word). The MapWritable serialization overhead is amortized over more co-occurrences. However, overall Stripes is slower than Pairs on this dataset because the large `MapWritable` values are expensive to serialize and deserialize.

1.4 Part (e): Local Aggregation

1.4.1 Overview

Local aggregation reduces the amount of data sent to the reducer by buffering and summing counts before emitting. Two levels of aggregation are implemented:

- **Map-Function Level:** Buffer is local to a single `map()` call. Counts are aggregated within one line and emitted after processing that line.
- **Map-Class Level:** Buffer lives for the entire lifetime of the mapper. Counts are aggregated across ALL lines and emitted only in `cleanup()`.

1.4.2 Map-Function Level Pseudocode

Listing 4: Map-Function Level Aggregation

```
map(key, line, context):
    words = filterWords(line)
    localBuffer = HashMap()   // local to THIS call only
    for i in range(len(words)):
        for j in neighbors within distance d:
            localBuffer["(words[i],words[j])"] += 1
            localBuffer["(words[j],words[i])"] += 1
    // Emit after processing whole line
    for pair, count in localBuffer:
        emit(pair, count)
```

1.4.3 Map-Class Level Pseudocode

Listing 5: Map-Class Level Aggregation

```
globalBuffer = HashMap()   // lives for entire mapper

setup(context):
```

```
    globalBuffer.clear()

map(key, line, context):
    words = filterWords(line)
    // Only update buffer, NEVER emit here
    for pair in generatePairs(words, d):
        globalBuffer[pair] += 1

cleanup(context):
    // Emit everything ONCE after all lines processed
    for pair, count in globalBuffer:
        emit(pair, count)
```

1.4.4 Runtime Comparison — Pairs with Local Aggregation

Method	d=1	d=2	d=3	d=4
Pairs (Basic)	5m 19s	5m 56s	7m 10s	9m 58s
Pairs (Map-Fn)	7m 11s	5m 17s	7m 52s	8m 37s
Pairs (Map-Class)	7m 38s	6m 41s	8m 23s	8m 16s

Table 6: Runtime comparison for Pairs approach with local aggregation

1.4.5 Runtime Comparison — Stripes with Local Aggregation

Method	d=1	d=2	d=3	d=4
Stripes (Basic)	11m 41s	10m 00s	8m 57s	8m 44s
Stripes (Map-Fn)	9m 04s	6m 00s	6m 01s	7m 01s
Stripes (Map-Class)	6m 33s	5m 40s	6m 07s	7m 23s

Table 7: Runtime comparison for Stripes approach with local aggregation

1.4.6 Analysis

Pairs Observations:

- Basic Pairs is fastest at d=1, but Map-Fn and Map-Class close the gap at higher d values.
- Local aggregation helps more at higher distances because more pairs are generated per line, giving more opportunity to aggregate before emitting.
- Map-Class level aggregation buffers more aggressively but adds overhead from maintaining a large global HashMap.
- The overhead of maintaining the HashMap sometimes outweighs the benefit on a single-node setup.

Stripes Observations:

- Local aggregation significantly benefits the Stripes approach. Map-Class Stripes is consistently faster than basic Stripes.
- Map-Class Stripes achieves the best performance because it maximally reduces the number of `MapWritable` objects emitted to the reducer.
- The benefit is more pronounced for Stripes than Pairs because `MapWritable` serialization is expensive — reducing emissions has a larger impact.

Overall: On this single-node setup, local aggregation is most beneficial for the Stripes approach. For Pairs, the benefit is less clear due to the overhead of HashMap management on a single node where there is no true parallel shuffle.

2 Problem 2: Indexing Documents via Hadoop

2.1 Part (a): Document Frequency

2.1.1 Program Logic

`DocumentFrequency.java` computes, for each distinct stemmed term, the number of documents in which it appears (Document Frequency). The Porter Stemmer from `opennlp-tools-1.9.3.ja` is used for stemming.

2.1.2 Pseudocode

Listing 6: DocumentFrequency Mapper

```
setup(context):
    Load stopWords from local file
    stemmer = new PorterStemmer()

map(key, line, context):
    tokens = tokenize(line)
    seenTerms = HashSet()   // ensures uniqueness per document
    for token in tokens:
        if len(token) > 2 AND token NOT in stopWords:
            stemmed = stemmer.stem(token)
            if stemmed NOT in seenTerms:
                seenTerms.add(stemmed)
                emit(stemmed, 1)

reduce(term, counts, context):
    emit(term, sum(counts))
```

2.1.3 Key Design Decisions

- A `HashSet<String> seenTerms` is used per document to ensure each term is counted *at most once per document* — this is the correct definition of Document Frequency.
- Porter Stemmer normalizes words (e.g., "running" → "run", "cities" → "citi") so that morphological variants are counted together.

- Stopwords are filtered in the mapper before stemming.
- A Combiner is used for partial aggregation.

2.1.4 Top 100 Terms by Document Frequency

Rank	Term (Stemmed)	DF	Rank	Term (Stemmed)	DF
1	apo	9518	26	follow	5615
2	categori	9381	27	three	5601
3	refer	9340	28	american	5501
4	quot	8674	29	earli	5473
5	other	8415	30	both	5454
6	link	8296	31	work	5427
7	thi	8190	32	origin	5402
8	extern	8008	33	through	5381
9	two	7949	34	present	5378
10	includ	7885	35	nation	5378
11	year	7801	36	univers	5194
12	first	7689	37	found	5139
13	http	7418	38	citi	5134
14	time	7349	39	form	5121
15	state	7299	40	make	5067
16	name	6838	41	end	5040
17	us	6823	42	area	5013
18	www	6804	43	dai	4949
19	see	6761	44	becam	4897
20	histori	6662	45	base	4893
21	peopl	6626	46	number	4857
22	part	6504	47	hous	4842
23	unit	6500	48	second	4839
24	made	6332	49	high	4836
25	known	6085	50	major	4832
(Top 50 shown; full top 100 available in `top100df.txt`)					

Table 8: Top 50 of 100 terms by Document Frequency (Porter Stemmed)

2.1.5 Runtime

Job	Runtime
DocumentFrequency (10,000 articles)	8m 16s

2.1.6 HDFS Commands

```
time hadoop jar DocumentFrequency.jar DocumentFrequency \
    /wiki/input /wiki/output2a

# Extract top 100 by DF
hdfs dfs -cat /wiki/output2a/part-r-00000 \
```

```
    | sort -t$'\t' -k2 -rn | head -100 | tee top100df.txt
```

2.2 Part (b): TF-IDF Score

2.2.1 Program Logic

`TFIDFScore.java` computes TF-IDF scores for each (document, term) combination using the top 100 terms identified in Part (a). It implements a Stripes approach as recommended by the assignment.

2.2.2 Formula

$$\text{SCORE} = TF \times \log\left(\frac{10000}{DF} + 1\right)$$

where TF is the term frequency within the document and DF is the document frequency from Part (a).

2.2.3 Pseudocode

Listing 7: TFIDFScore Mapper

```
setup(context):
    Load stopWords from local file
    Load dfMap (term -> DF) from top100df.txt
    stemmer = new PorterStemmer()

map(key, line, context):
    docId = filename of input split (without .txt)
    tokens = tokenize(line)
    tfMap = HashMap()    // term -> TF count
    for token in tokens:
        if token not in stopWords:
            stemmed = stemmer.stem(token)
            if stemmed in dfMap:
                tfMap[stemmed] += 1
    // Compute and emit TF-IDF score
    for term, tf in tfMap:
        df = dfMap[term]
        score = tf * log(10000.0 / df + 1)
        emit(docId, term + "\t" + score)

reduce(docId, values, context):
    for val in values:
        term, score = val.split("\t")
        emit(docId, term + "\t" + score)
```

2.2.4 Key Design Decisions

- Only the top 100 terms (from Part a) are scored — all other terms are ignored.

- The document ID is derived from the input filename (e.g., `100005.txt` → `100005`).
- The DF file (`top100df.txt`) is loaded into memory in `setup()` — this is the Stripes approach for handling the small (100-term) DF lookup table.
- The formula uses `10000.0` (not `10000`) to ensure floating-point division.

2.2.5 Sample Output

```
100005  first     0.8332
100005  time      1.7179
100005  refer     0.7279
100005  thi       6.3837
100005  form      3.2482
100005  june      1.1999
100005  earli     9.3534
100005  categori  1.4512
```

2.2.6 Runtime

Job	Runtime
TFIDFScore (10,000 articles, 100 terms)	5m 51s

2.2.7 HDFS Commands

```
time hadoop jar TFIDFScore.jar TFIDFScore \
    /wiki/input /wiki/output2b

# View output
hdfs dfs -cat /wiki/output2b/part-r-00000 | head -20
```

3 Overall Runtime Summary

Method	d=1	d=2	d=3	d=4
Pairs (Basic)	5m 19s	5m 56s	7m 10s	9m 58s
Pairs (Map-Fn)	7m 11s	5m 17s	7m 52s	8m 37s
Pairs (Map-Class)	7m 38s	6m 41s	8m 23s	8m 16s
Stripes (Basic)	11m 41s	10m 00s	8m 57s	8m 44s
Stripes (Map-Fn)	9m 04s	6m 00s	6m 01s	7m 01s
Stripes (Map-Class)	6m 33s	5m 40s	6m 07s	7m 23s
Problem 2a (DocumentFrequency)		8m 16s		
Problem 2b (TFIDFScore)		5m 51s		

Table 9: Complete runtime summary — all experiments on 10,000 Wikipedia articles (single-node Hadoop, macOS)

4 Conclusions

1. **Pairs vs Stripes:** On this dataset, basic Pairs outperforms basic Stripes for small d. This is because `MapWritable` serialization in Stripes adds significant overhead on a single node. However, as d increases, Stripes becomes more competitive because the number of emitted keys stays constant while only the map values grow.
2. **Local Aggregation:** Local aggregation is more beneficial for Stripes than Pairs. For Stripes, Map-Class aggregation reduces the number of expensive `MapWritable` emissions significantly, leading to consistent speedups. For Pairs, the benefit is less clear due to HashMap overhead on a single node.
3. **Map-Class vs Map-Function:** Map-Class aggregation is theoretically more powerful (buffers across all lines) but can be memory-intensive for large datasets. Map-Function aggregation is safer but provides less reduction in emitted records.
4. **Distance Effect:** For Pairs, runtime grows with d because more pairs are generated. For Stripes, the effect is smaller because the number of emitted keys is fixed — only value sizes grow.
5. **TF-IDF:** The Porter Stemmer effectively normalizes terms, and the 10,000-document normalization constant in the TF-IDF formula provides meaningful relative scores across documents.